

35.17. La vitesse de l'avion : une variable, vitesse

Une variable, *vitesse*, dit combien de pixels l'avion parcourt à chaque image : 1 lent, 2 normal, 4 très rapide. Les bords se calculent avec elle.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : une variable `vitesse`. Au 35.16, l'avion avançait toujours d'un pixel par image (`px++`). Maintenant il avance de `vitesse` pixels : `px = px + vitesse;`. Pour changer sa vitesse, on change **un seul nombre**, en haut du programme.

Ce que vaut chaque réglage : l'écran se redessine 60 fois par seconde. `vitesse = 1` → 60 pixels par seconde (le 35.16) ; `2` → **120** (il traverse l'écran en un peu plus d'une seconde) ; `4` → **240**, très rapide. Au-delà, le vol devient saccadé : l'œil voit l'avion sauter.

Pourquoi les bords changent : avec `vitesse = 2`, l'ancien test `px > 0` ne suffit plus. Si `px` vaut **1**, il est plus grand que 0, et `px - 2` donnerait... pas -1 : un `uint8_t` ne descend **jamais sous 0**, il **repart à 255**. L'avion sauterait à l'autre bout de l'écran.

Les nouveaux tests : on vérifie **avant** de bouger que le pas tient. À gauche : `px >= vitesse` (il reste au moins un pas avant 0). À droite : `px + vitesse <= 144` (après le pas, il est encore entier à l'écran). Pareil en hauteur, avec 128. Ces tests marchent **quelle que soit** la vitesse.

Déroulé, vitesse 2, flèche gauche tenue depuis `px = 4` : $4 \geq 2$, donc `px = 2` ; $2 \geq 2$, donc `px = 0` ; $0 \geq 2$ est faux : il s'arrête, **pile au bord**.

À toi : essaie `vitesse = 1`, puis `4`. Avec `5`, regarde où il s'arrête : $72 + 5 + 5 + \dots$ ne tombe pas juste sur 144 — il s'arrête à **142**, le dernier pas qui tient ($142 + 5 = 147$, trop loin). À gauche, il s'arrête à **2** ($2 \geq 5$ est faux) : jamais de saut à 255.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <sprite16>` → [35.2. La fonction `sprite16\(\)` — un personnage de \$16 \times 16\$](#)
- `#include <bouton>` → [0.65.5. La fonction `bouton\(\)` — lire la manette](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **497 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	59 o	← la console, à l'allumage — la mise en route de la console
main()	218 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde

AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
RangerLutins	11 o	← démarrage	– range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage	– installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	64 o	← CopierTuiles	– 0 lettre sur 43, et 4 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert	– la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

sprite()	× 4	104 o	sur place, dans main()
bouton()	× 4	20 o	sur place, dans main() + LireManette (37 o)
image()	× 1	6 o	sur place, dans main() + AttendreImage (28 o)
sprite16()	× 1	0 o	sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// La vitesse de l'avion : un seul nombre à changer, en haut.

#include "personnages.cpp" // la liste : AVION
#include <sprite16>        // sprite16() : quatre lutins en carré
#include <bouton>           // bouton() : lit un bouton de la manette
```

2. UNE VARIABLE — VITESSE

```
uint8_t vitesse = 2; // pixels par image : 1 = lent, 2 = normal, 4 = très rapide
```

3. UNE VARIABLE — PX

```
uint8_t px = 72; // la place de l'avion, en pixels : de 0 à 144 (160 - 16)...
```

4. UNE VARIABLE — PY

```
uint8_t py = 64; // ...et de 0 à 128 (144 - 16). Il part du milieu.
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

        // On vérifie AVANT de bouger que le pas tient dans l'écran.
        if (bouton(DROITE) && px + vitesse <= 144) px = px + vitesse;
        if (bouton(GAUCHE) && px >= vitesse)      px = px - vitesse;
        if (bouton(BAS) && py + vitesse <= 128)   py = py + vitesse;
        if (bouton(HAUT) && py >= vitesse)        py = py - vitesse;

        sprite16(0, px, py, AVION); // l'avion à sa place : lutins 0 à 3
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



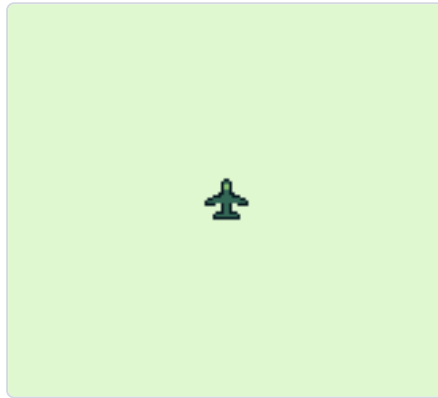
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

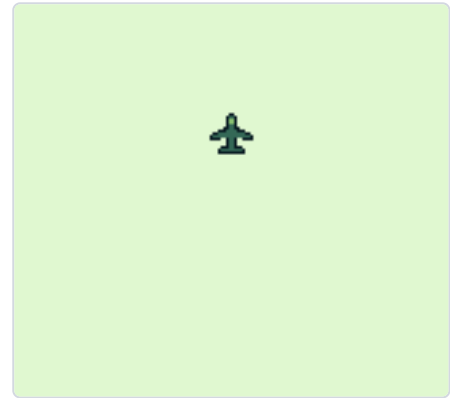
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



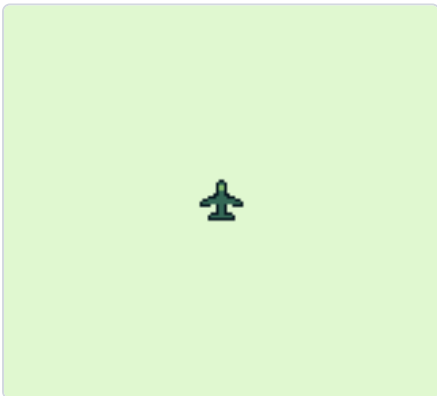
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le même avion, deux fois plus rapide qu'au 35.16 : il traverse l'écran en un peu plus d'une seconde, et s'arrête toujours pile au bord.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ vitesse 2 : il va deux fois plus loin qu'au 35.16 (px = 112)

✓ à droite, il s'arrête pile à 144

✓ à gauche, pile à 0 — sans repartir à 255

✓ en haut, pile à 0

✓ le lutin suit sa place

Ensuite : **35.18. Changer de vitesse en vol : A accélère, B ralentit** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++